

Raport z realizacji prototypu w ramach projektu badawczego „PlasmaRand”

Projekt realizowany w ramach studenckiego koła naukowego *Wirtualnego Latania vFly*. Celem projektu jest opracowanie systemu generowania prawdziwie losowych liczb, opartego na działaniu lamp plazmowych.

Lampy plazmowe są obserwowane przez kamerę, której zadaniem jest rejestrowanie losowych wyładowań elektrycznych. Zarejestrowana klatka obrazu jest konwertowana na ciąg bitów stanowiący wejście dla algorytmu haszującego. Wynikowa wartość służy jako seed dla generatora, co pozwala na uzyskanie liczb o wysokim stopniu losowości.

- ***Implementacja algorytmów z użyciem języka Java z zastosowaniem wzorców projektowych***

Aplikacja została podzielona na trzy odseparowane warstwy zgodnie z wzorcem architektonicznym Model-Widok-Kontroler, co zapewnia czytelność kodu i łatwość utrzymania. Tym samym logika domenowa jest całkowicie niezależna od interfejsu użytkownika, a komunikacją między nimi zarządza kontroler.

W projekcie zastosowano wzorec Strategii do obsługi wszystkich zaimplementowanych algorytmów haszujących oraz generatorów liczb losowych. Pozwoliło to na dynamiczną wymianę algorytmu bez modyfikacji kodu klienta.

Do implementacji generatorów liczb losowych wykorzystano klasy `SecureRandom`, `Security` oraz `DrbgParameters` pochodzące z biblioteki `Java Security`. Z tej samej biblioteki pochodzi klasa `MessageDigest`, którą wykorzystano przy implementacji algorytmów kryptograficznych, takich jak SHA-256, czy SHA-512. Algorytmy haszujące wykorzystujące prostą arytmetykę np. DJB2, Knuth przygotowano w oparciu o standardowe operacje bitowe. Wykorzystano również wzorec projektowy `Dependency Injection`. Obiekty nie tworzą swoich zależności samodzielnie, lecz otrzymują je w konstruktorze.

Zastosowano także wzorec `Repository` w celu zwiększenia abstrakcji warstwy dostępu do danych.

- ***Opracowanie graficznego interfejsu użytkownika***

Zaimplementowano interfejs desktopowy przy użyciu biblioteki `Java Swing`. Aplikacja posiada główne okno, które dziedziczy po `JFrame`. Wykorzystano menedżera układu do rozmieszczenia komponentów. Zaimplementowano obsługę zdarzeń dla przycisków oraz interakcję z systemem plików.

- ***Usprawnianie oprogramowania stosując mechanizmy podnoszące poziom abstrakcji oraz wykorzystując zaawansowane struktury danych***

Kluczowe komponenty systemu zostały zdefiniowane za pomocą interfejsów. Dzięki temu kod wyższego poziomu nie jest uzależniony od konkretnych implementacji, lecz od abstrakcji.

Wydzielono logikę przetwarzania wideo do klasy VideoGrabber, która ukrywa operacje biblioteki FFmpeg.

Zastosowanie rekordów pozwoliło na uproszczenie przepływu danych oraz znaczą redukcję powtarzalnego kodu (tzw. boilerplate).

Struktury danych: Wykorzystano m.in. HashMap do przechowywania dostępnych strategii oraz ArrayList do buforowania wyników przetwarzania wideo przed zapisem do bazy.

- ***Przeprowadzanie kompletnej weryfikacji oprogramowania wykorzystując testy jednostkowe***

Zastosowano framework JUnit do testowania logiki aplikacji. Testy podzielono na trzy poziomy:

- Testy Algorytmów: Weryfikują poprawność matematyczną funkcji skrótu.
- Testy Serwisu: Weryfikują integrację komponentów, np. czy metoda processVideo poprawnie iteruje po klatkach i czy obsługuje błędy
- Testy Repozytorium: Weryfikują warstwę dostępu do danych. Sprawdzają poprawność zapytań SQL. Testy te upewniają się, że obiekty aplikacji są poprawnie mapowane na rekordy w tabeli bazy danych oraz czy operacje zapisu nie powodują błędów naruszenia integralności danych.

- ***Opracowanie prostej aplikacji WWW z wykorzystaniem technologii Java***

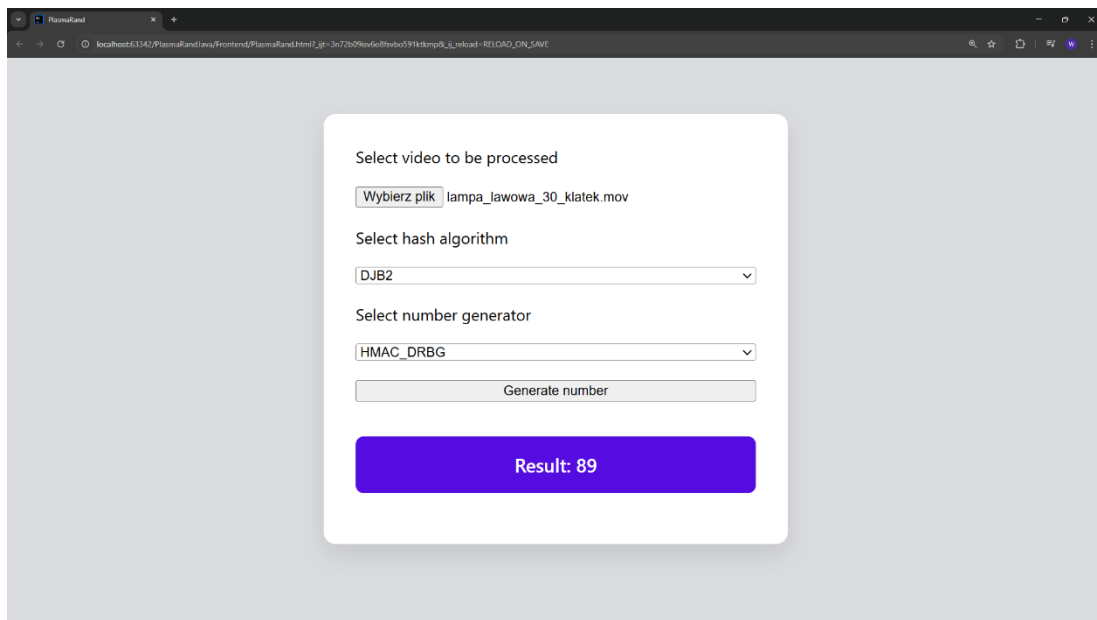
Projekt umożliwia uruchomienie aplikacji w trybie webowym (z wykorzystaniem wbudowanego serwera Tomcat). Logika aplikacji jest współdzielona z wersją desktopową, co świadczy o poprawnym odseparowaniu warstw. Wykorzystano Servlety do obsługi żądań HTTP. Początkowo interfejs użytkownika oparto na technologii JSP z wykorzystaniem biblioteki JSTL, jednak w trakcie rozwoju projektu dokonano refaktoryzacji na rzecz asynchronicznego JavaScripta.

- **Implementacja elementu bazy danych z wykorzystaniem technologii Java**

Zaimplementowano trwałość danych przy użyciu bazy SQLite. Zastosowano wzorzec Repository, aby odseparować logikę aplikacji od logiki dostępu do danych. Aplikacja zapisuje wygenerowane wyniki (użyte funkcje hashujące, hashe, liczby, czasy i datę wykonania).

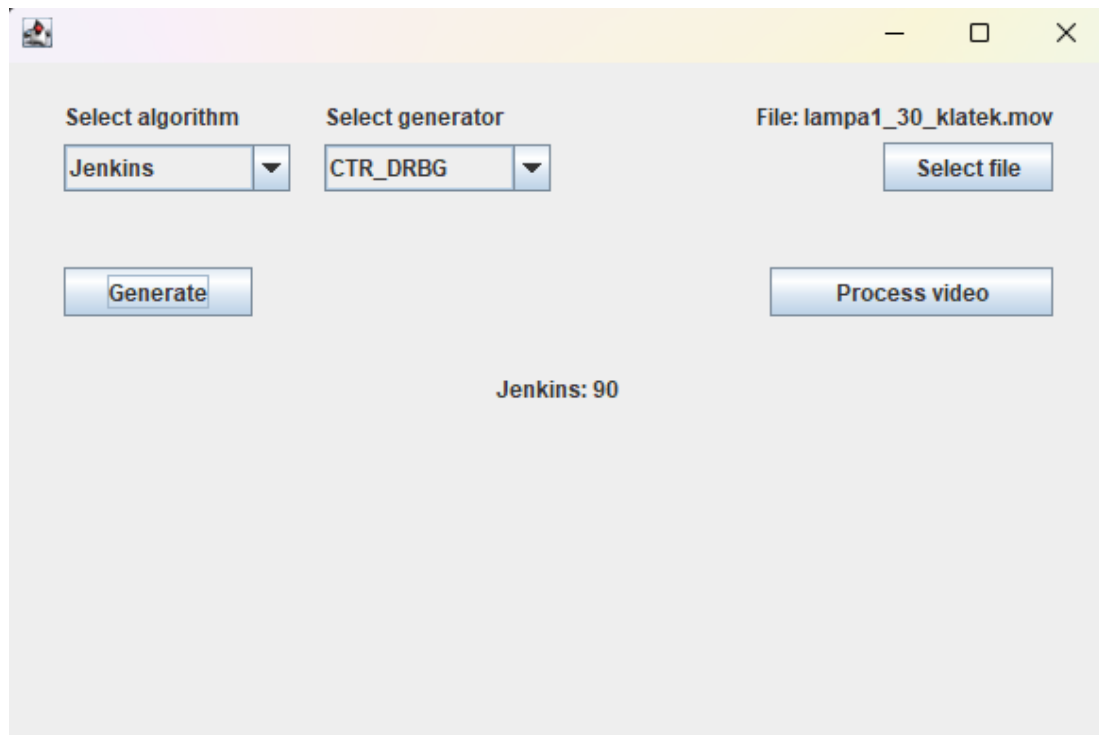
Poniżej przedstawiono końcowy wygląd interfejsu użytkownika w dwóch zaimplementowanych wersjach:

- **Wersja webowa**



Rysunek 1. Interfejs webowy aplikacji, uruchomionej na wbudowanym serwerze Tomcat

- Wersja desktopowa



Rysunek 2. Graficzny interfejs użytkownika zrealizowany przy użyciu biblioteki Java Swing